

Tarskii Project Primer

Spiking Neural Networks in Hardware

From Brain to Silicon: An Introduction

Project Tarskii

27th January 2026

About this document: This primer provides background material for understanding the Tarskii project, a neuromorphic computing initiative aimed at developing ultra-low-power AI hardware. We cover the fundamentals of neural networks, spiking neural networks, and the mathematics behind training them, before introducing our simulator, Gilgamesh, and discussing the energy challenges that motivate this work.

Contents

1 What is a Neural Network?

At its core, a neural network is a computational system loosely inspired by biological brains. The idea is simple: connect many simple processing units together, and complex behaviour emerges from how they interact.

1.1 The Biological Inspiration

The human brain contains approximately 86 billion neurons, each connected to thousands of others through synapses. When a neuron receives enough input from its neighbours, it “fires”, sending an electrical signal to all the neurons it connects to. This basic mechanism, repeated across billions of units, gives rise to everything from vision to language to consciousness.

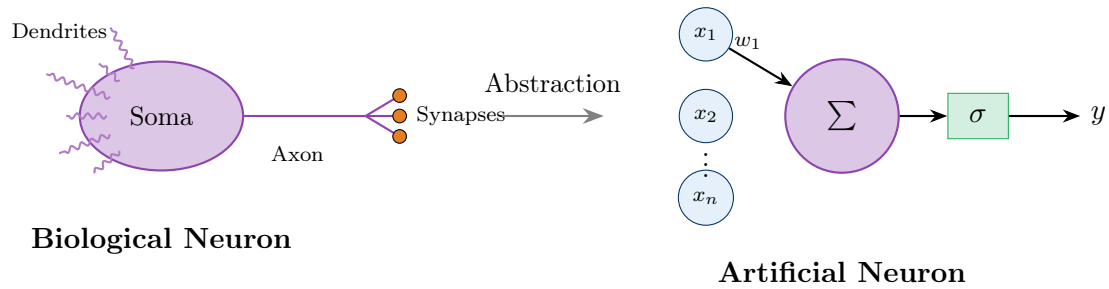


Figure 1: From biology to mathematics: a biological neuron and its artificial counterpart.

1.2 Artificial Neural Networks

An artificial neural network takes this biological metaphor and turns it into mathematics. Each artificial neuron:

1. Receives inputs from other neurons (or from external data)
2. Multiplies each input by a **weight** (representing synaptic strength)
3. Sums all the weighted inputs
4. Applies an **activation function** to produce an output

Mathematically, a single neuron computes:

$$y = \sigma \left(\sum_{i=1}^n w_i x_i + b \right) \quad (1)$$

where x_i are inputs, w_i are weights, b is a bias term, and σ is the activation function.

Key Difference: Biological neurons communicate through discrete *spikes*, brief electrical pulses. Traditional artificial neural networks (ANNs) use continuous values instead. This distinction becomes important when we discuss spiking neural networks later.

1.3 Brain vs. Artificial Networks: A Comparison

Aspect	Brain	Modern LLMs
Number of neurons/parameters	~86 billion	1-2+ trillion
Connections per neuron	~7,000	Varies (attention spans all tokens)
Communication	Discrete spikes	Continuous values
Processing	Massively parallel	Parallel (GPU clusters)
Power consumption	~20 watts	100s of kW (inference clusters)
Precision	Noisy, variable	16/32-bit floating point
Learning	Continuous, local	Batched, global (backprop)

Table 1: Comparison between biological and modern artificial neural networks.

The brain's energy efficiency is striking: it performs complex cognition on roughly the power of a dim light bulb. This is a key motivation for neuromorphic computing.

2 ANNs vs SNNs: Two Paradigms

There are two different approaches to neural computation: Artificial Neural Networks (ANNs) and Spiking Neural Networks (SNNs). Understanding the distinction matters for this project.

2.1 Artificial Neural Networks (ANNs)

ANNs, the workhorses of modern deep learning, process information in discrete layers. Information flows forward through the network, with each layer transforming its inputs through weighted sums and activation functions. The key characteristic: neurons output *continuous values*.

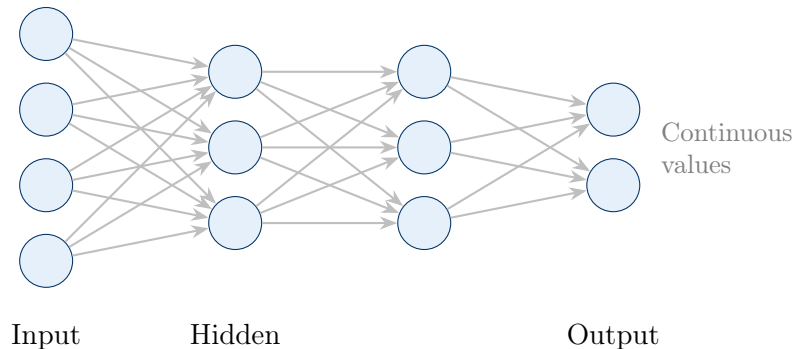


Figure 2: A feedforward ANN: information flows from input to output through hidden layers.

2.2 Spiking Neural Networks (SNNs)

SNNs take a different approach. Rather than passing continuous values, neurons communicate through *spikes*: discrete, all-or-nothing events. This mirrors biological neurons more closely, but introduces a new dimension: **time**.

In an SNN:

- Neurons maintain an internal state (membrane potential)
- Incoming spikes cause this potential to rise

- When potential crosses a threshold, the neuron fires its own spike
- After firing, the potential resets

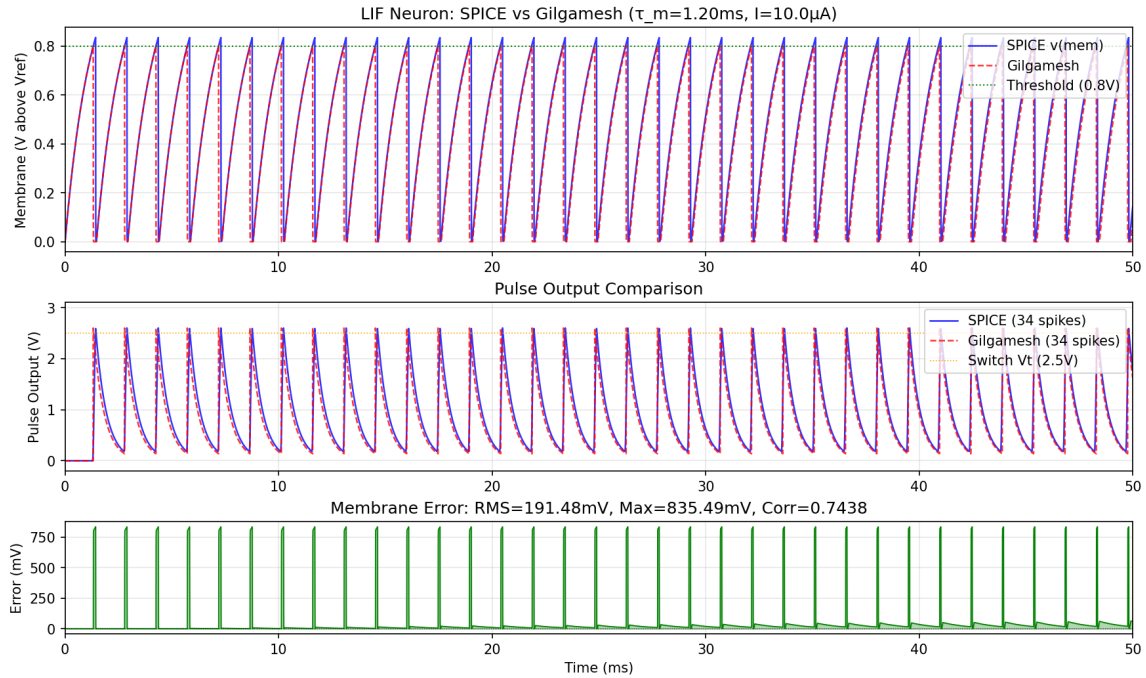


Figure 3: Gilgamesh vs ngspice validation: membrane potential traces (top), pulse outputs (middle), and error analysis (bottom). Both simulators produce 34 matching spikes, validating our physics model.

2.3 Key Differences

Aspect	ANN	SNN
Communication	Continuous values	Binary spikes
Temporal dynamics	None (static)	Explicit time dimension
Computation	Multiply-accumulate	Event-driven
Training	Backpropagation (easy)	More challenging
Hardware efficiency	Power-hungry	Potentially very efficient
Biological realism	Low	Higher

Why SNNs for hardware? The event-driven nature of SNNs means computation only happens when spikes occur. In sparse networks, this can reduce energy consumption by orders of magnitude: a neuron that isn't spiking consumes almost no power.

3 The Leaky Integrate-and-Fire Neuron

The Leaky Integrate-and-Fire (LIF) neuron is the workhorse of computational neuroscience and neuromorphic engineering. It captures the core dynamics of biological neurons while remaining simple enough to simulate efficiently and implement in hardware.

3.1 The Model

The LIF neuron treats the cell membrane as a leaky capacitor. Input current charges the membrane, but charge also leaks away over time. The dynamics are governed by:

$$\tau_m \frac{dV}{dt} = -V + R \cdot I(t) \quad (2)$$

where:

- V is the membrane potential
- $\tau_m = RC$ is the membrane time constant
- R is the membrane resistance
- $I(t)$ is the input current

When V crosses a threshold θ , the neuron emits a spike and resets.

3.2 Circuit Analogy

The LIF neuron has a direct circuit interpretation, which makes it well-suited for hardware implementation:

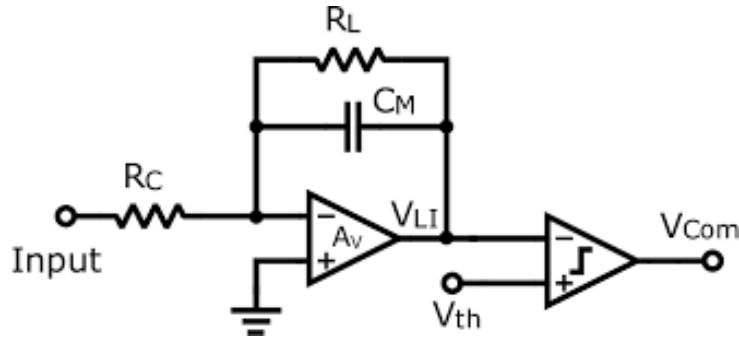


Figure 4: Our LIF neuron circuit implementation. The RC network (R_L , C_M) integrates synaptic current while providing natural leakage. The op-amp buffer isolates the membrane voltage, and the comparator detects threshold crossings to generate spikes.

3.3 Discrete-Time Implementation

For simulation and training, we discretise the continuous dynamics. With timestep Δt :

$$V[t+1] = \beta \cdot V[t] + I[t] \quad (3)$$

where $\beta = e^{-\Delta t/\tau_m}$ is the decay factor (typically ~ 0.9).

The spike generation and reset:

$$S[t] = \mathcal{H}(V[t] - \theta) \quad (4)$$

$$V_{reset}[t] = V[t] - \theta \cdot S[t] \quad (5)$$

where $\mathcal{H}$ is the Heaviside step function.

Typical Parameters:

- Membrane time constant: $\tau_m \approx 4\text{--}10$ ms
- Threshold: $\theta = 1.0$ (normalised)
- Decay factor: $\beta = 0.9$
- In our hardware: $R = 120\text{ k}\Omega$, $C = 33\text{ nF}$, giving $\tau_m \approx 4\text{ ms}$

4 ngspice: Circuit Simulation

ngspice is an open-source circuit simulator, the modern descendant of SPICE (Simulation Program with Integrated Circuit Emphasis), originally developed at Berkeley in the 1970s. It has become the industry standard for analog and mixed-signal circuit simulation.

4.1 Why ngspice?

For neuromorphic hardware development, ngspice serves a critical role: it lets us verify that our abstract neuron models match the behaviour of real transistor-level circuits.

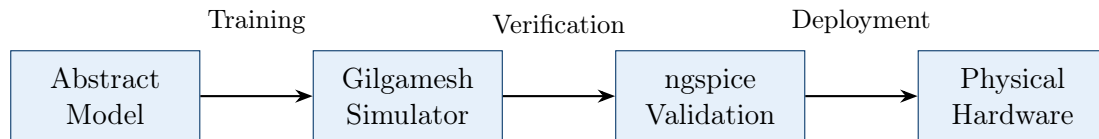


Figure 5: The development pipeline: from abstract models to verified hardware.

4.2 What ngspice Does

ngspice solves the differential equations that govern electronic circuits:

- **Transient analysis:** Simulates circuit behaviour over time
- **DC analysis:** Finds steady-state operating points
- **AC analysis:** Analyses frequency response
- **Monte Carlo:** Models component variations and manufacturing tolerances

For our neurons, ngspice can simulate the exact voltage waveforms, accounting for:

- Parasitic capacitances and resistances
- Transistor non-idealities
- Temperature variations
- Component tolerances

4.3 Gilgamesh-ngspice Integration

We use ngspice to verify that Gilgamesh’s physics models accurately match transistor-level circuit behaviour. This validation process involves:

1. Simulating individual LIF neurons in both Gilgamesh and ngspice
2. Comparing membrane voltage traces and spike timing
3. Measuring error metrics (RMS error, correlation)

4. Iterating on Gilgamesh’s physics parameters until they match

This ensures that when we move to hardware, the behaviour we trained for in simulation will actually occur in the physical circuit.

5 Rust: The Implementation Language

Gilgamesh is written in **Rust**, a systems programming language increasingly used for performance-critical applications. But why Rust for a neural network simulator?

5.1 Key Features

Memory safety without garbage collection: Rust’s ownership system prevents entire classes of bugs (null pointer dereferences, buffer overflows, data races) at compile time. This matters when simulating thousands of neurons over millions of timesteps.

Zero-cost abstractions: High-level code compiles to efficient machine code. We can write expressive, maintainable code without sacrificing performance.

Fearless concurrency: Rust’s type system prevents data races, making it safe to parallelise across neurons and timesteps.

C/C++ interoperability: We can call existing libraries (like BLAS for linear algebra) when needed.

5.2 Performance Comparison

Language	Speed	Memory Safety	Ease of Use
C/C++	Excellent	Manual	Moderate
Python	Poor	Automatic	Excellent
Rust	Excellent	Automatic	Moderate

Table 2: Rust offers the performance of C with the safety of managed languages.

5.3 Example: LIF Forward Pass

Here’s a taste of how Gilgamesh implements the LIF forward pass in Rust:

Listing 1: Simplified LIF forward pass in Rust

```

1 fn forward(&self, input: &Array1<f32>, mem: &mut Array1<f32>)
2   -> Array1<f32> {
3     // Update membrane potential
4     *mem = &*mem * self.beta + input;
5
6     // Generate spikes where threshold exceeded
7     let spikes = mem.mapv(|v| if v > self.threshold { 1.0 } else { 0.0
8       });
9
10    // Reset spikd neurons
11    *mem = mem - &spikes * self.threshold;
12
13    spikes
  }
```

The code is concise, readable, and compiles to highly optimised machine code.

6 Gilgamesh: Our SNN Simulator

Gilgamesh is a hardware-accurate spiking neural network simulator developed as part of the Tarskii project. Named after the ancient Mesopotamian king, Gilgamesh bridges the gap between abstract neural network models and physical neuromorphic hardware.

6.1 Design Philosophy

Gilgamesh operates in two modes:

Simple Mode: Fast simulation compatible with frameworks like `snnTorch`. Uses discrete-time dynamics with a decay factor β . Good for rapid prototyping and training.

Physics Mode: Accurate RC-circuit dynamics matching real hardware. Uses physical parameters (resistance, capacitance, voltage rails). Required for hardware validation.

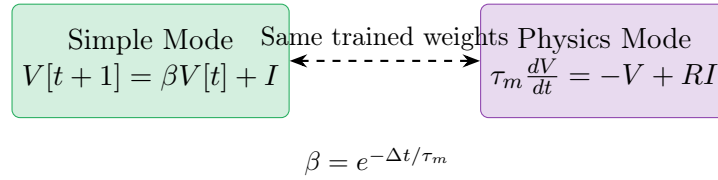


Figure 6: Gilgamesh’s dual-mode architecture: train fast, validate accurately.

6.2 Network Architecture

Our hardware-optimised network for MNIST classification uses a compact 36-12-10 architecture:

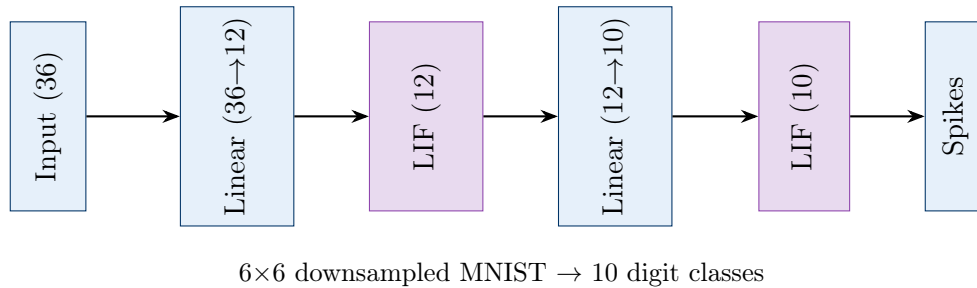


Figure 7: Gilgamesh network architecture. LIF neurons are shown in purple.

Total synapses: 552 ($36 \times 12 + 12 \times 10$)

Achieved accuracy: 90.69% on MNIST test set

This architecture was chosen through iterative search, balancing accuracy against hardware complexity. Beyond 90% accuracy, the additional synapses required yield diminishing returns for our target applications.

6.3 Key Features

- **Hardware parameter mapping:** Converts normalised weights to physical currents and voltages

- **SPICE validation:** Generate ngspice simulations to verify physics models
- **Multiple surrogate gradients:** FastSigmoid, ATan, Sigmoid, Straight-Through
- **Physics effects:** Pulse stretching, threshold adaptation, comparator delays
- **Quantisation support:** 4-bit weights for current source DAC implementation
- **Visualisation:** Real-time dashboards for spike activity

7 The Mathematics of Gilgamesh

This section details the mathematical foundations underpinning Gilgamesh.

7.1 LIF Dynamics

7.1.1 Simple Mode

$$V[t + 1] = \beta \cdot V[t] + I[t] \quad (6)$$

$$S[t] = \mathcal{H}(V[t] - \theta) \quad (7)$$

$$V_{reset}[t] = V[t + 1] - \theta \cdot S[t] \quad (8)$$

7.1.2 Physics Mode

$$\frac{dV}{dt} = \frac{I_{syn}}{C} - \frac{V}{\tau_m} \quad (9)$$

$$\text{Discretised: } V[t + 1] = I \cdot \tau_m \cdot (1 - e^{-\Delta t / \tau_m}) + V[t] \cdot e^{-\Delta t / \tau_m} \quad (10)$$

The connection: $\beta = e^{-\Delta t / \tau_m}$, so physics mode reduces to simple mode when appropriately parameterised.

7.2 Spike Generation

The spike function is a Heaviside step:

$$S = \mathcal{H}(V - \theta) = \begin{cases} 1 & \text{if } V > \theta \\ 0 & \text{otherwise} \end{cases} \quad (11)$$

7.3 Loss Function

Classification uses cross-entropy on spike counts:

$$c_i = \sum_{t=1}^T S_i[t] \quad (\text{spike count for class } i) \quad (12)$$

$$p_i = \frac{e^{c_i}}{\sum_j e^{c_j}} \quad (\text{softmax}) \quad (13)$$

$$\mathcal{L} = -\log(p_{target}) \quad (14)$$

7.4 Optimisation

Gilgamesh uses Adam with weight decay (AdamW):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla \mathcal{L} \quad (15)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla \mathcal{L})^2 \quad (16)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (17)$$

$$w_{t+1} = w_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda w_t \right) \quad (18)$$

Default parameters: $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, $\lambda = 0.01$

8 Backpropagation and BPTT

Training neural networks requires computing how to adjust weights to reduce errors. This is done through **backpropagation**: propagating error signals backwards through the network.

8.1 Standard Backpropagation

For a feedforward network, backpropagation applies the chain rule layer by layer:

$$\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_i} \quad (19)$$

where z is the pre-activation and y is the output.

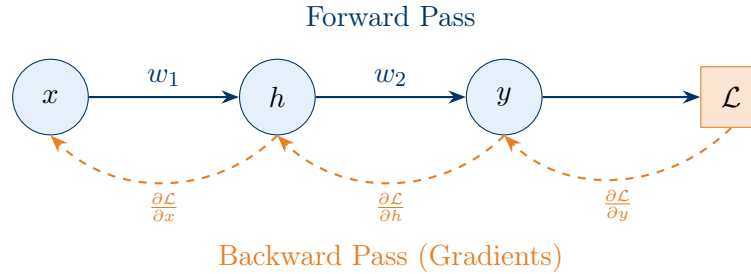


Figure 8: Backpropagation: gradients flow backward from the loss to update weights.

8.2 Backpropagation Through Time (BPTT)

SNNs have recurrent dynamics: the membrane potential at time t depends on time $t - 1$. This temporal dependency requires **Backpropagation Through Time** (BPTT).

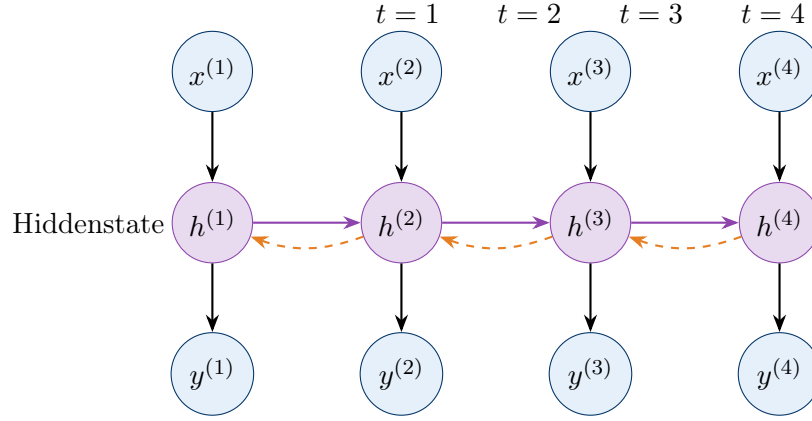


Figure 9: BPTT unrolls the network in time. Gradients (orange, dashed) flow backwards through both layers and timesteps.

The gradient through the membrane state:

$$\frac{\partial \mathcal{L}}{\partial V[t]} = \frac{\partial \mathcal{L}}{\partial S[t]} \cdot \frac{\partial S[t]}{\partial V[t]} + \frac{\partial \mathcal{L}}{\partial V[t+1]} \cdot \frac{\partial V[t+1]}{\partial V[t]} \quad (20)$$

The second term captures how past membrane states affect future outputs. This is the “through time” component.

The problem: $\frac{\partial S}{\partial V} = \frac{d\mathcal{H}}{dV}$ is zero everywhere except at the threshold, where it’s undefined. The Heaviside function has no useful gradient for learning!

9 Surrogate Gradients

The non-differentiability of spike generation is the central challenge of training SNNs. **Surrogate gradients** provide a practical workaround.

9.1 The Problem

The Heaviside step function $\mathcal{H}(x)$ has derivative:

$$\frac{d\mathcal{H}}{dx} = \begin{cases} 0 & \text{if } x \neq 0 \\ \text{undefined} & \text{if } x = 0 \end{cases} \quad (21)$$

With zero gradients, backpropagation cannot update weights. The network cannot learn.

9.2 The Solution

Use a smooth, differentiable function as a *surrogate* during the backward pass:

- **Forward pass:** Use the true Heaviside (binary spikes)
- **Backward pass:** Use a smooth approximation (non-zero gradients)

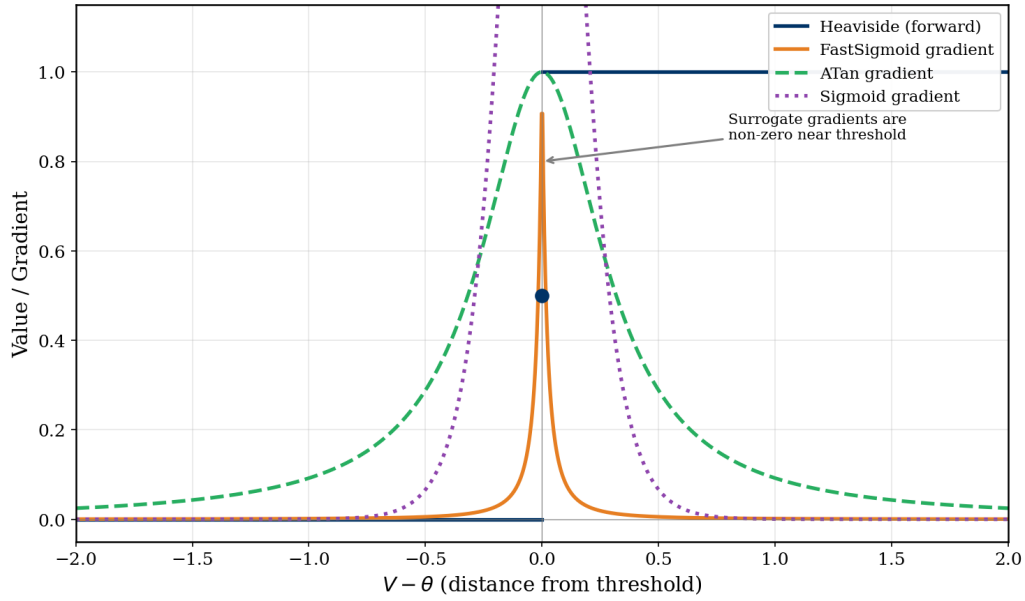


Figure 10: Surrogate gradients provide smooth, non-zero derivatives where the Heaviside has none. The peak at $V = \theta$ allows gradients to flow during backpropagation.

9.3 Surrogate Functions in Gilgamesh

FastSigmoid (SuperSpike), default, slope $k = 25$:

$$\sigma'(x) \approx \frac{1}{(k|x| + 1)^2} \quad (22)$$

ATan, snnTorch default, $\alpha = 2$:

$$\sigma'(x) \approx \frac{\alpha/2}{1 + (\pi\alpha x/2)^2} \quad (23)$$

Sigmoid, classical, slope k :

$$\sigma'(x) \approx \frac{k \cdot e^{-kx}}{(e^{-kx} + 1)^2} \quad (24)$$

Straight-Through Estimator:

$$\sigma'(x) = 1 \quad (\text{everywhere}) \quad (25)$$

Intuition: Surrogate gradients say “when the membrane is close to threshold, small changes matter a lot; when it’s far from threshold, they don’t.” The slope parameter controls how sharply this transition occurs.

10 Ireland's Data Centre Problem

Ireland has become a European hub for data centres, hosting facilities for Google, Amazon, Microsoft, Meta, and many others. But this success has created a significant energy challenge.

10.1 The Scale of the Problem

Key statistic: Data centres now consume approximately **21% of Ireland's total electricity**, more than all urban households combined.

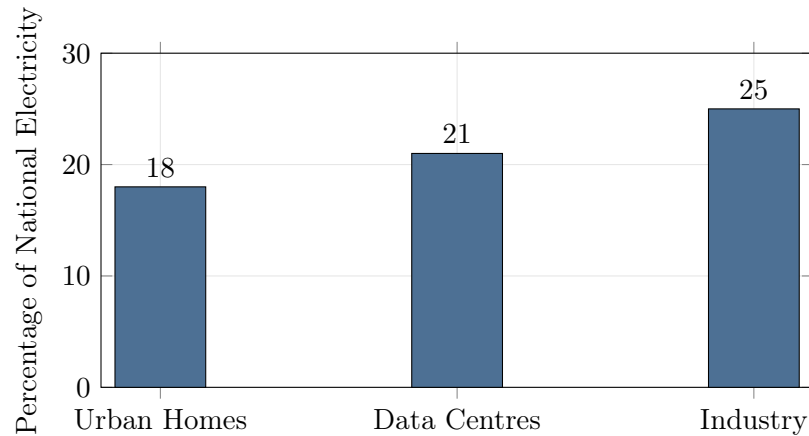


Figure 11: Ireland's electricity consumption by sector (approximate, 2024).

10.2 Why Ireland?

Several factors made Ireland attractive for data centres:

- Cool climate (reducing cooling costs)
- EU membership and data sovereignty
- English-speaking workforce
- Favourable corporate tax environment
- Strong renewable energy potential

10.3 The Grid Strain

EirGrid, Ireland's transmission system operator, has raised concerns:

- Data centres strain the grid during peak demand
- New connections have been restricted in the Dublin region
- Renewable energy targets become harder to meet
- Industrial users face higher electricity costs

10.4 AI Exacerbates the Problem

Modern AI workloads are particularly power-hungry:

- Training large language models can consume megawatts for weeks
- Inference at scale (serving millions of users) requires constant power
- GPU clusters generate significant heat, requiring energy-intensive cooling
- AI chip manufacturing is itself energy-intensive

11 AI's Energy Challenge

The explosive growth of AI has brought its energy consumption into sharp focus. While training costs grab headlines, **inference at scale** is where the real energy crisis lies.

11.1 The Inference Problem

Training happens once; inference happens billions of times per day. Consider Google's Gemini:

- Over **650 million monthly active users** as of late 2025
- **35–45 million daily active users** making queries
- **1.5 billion monthly AI Overview interactions** in Google Search alone
- Gemini 3 Pro is estimated at **2+ trillion parameters**, larger than models like Kimi K2 (1T parameters)

Each query to a model of this scale requires substantial computation, running on hardware like NVIDIA's H200 GPUs.

11.1.1 Napkin Maths: The Power Bill

Let's estimate the power consumption for AI inference at scale:

Assumptions (conservative):

- 40 million daily active users $\times$ 10 queries/day = **400 million queries/day**
- Average $\approx$ 4,600 queries/second, peak $\approx$ 15,000 queries/second
- A 2T parameter model needs $\sim$ 8 H200 GPUs per inference server
- Each server handles $\sim$ 10 queries/second

Working through the numbers:

$$\text{Servers needed (peak)} = \frac{15,000 \text{ queries/s}}{10 \text{ queries/s/server}} = 1,500 \text{ servers}$$

$$\text{GPUs} = 1,500 \times 8 = 12,000 \text{ H200s}$$

$$\text{GPU power} = 12,000 \times 700\text{W} = 8.4 \text{ MW}$$

$$\text{With cooling \& infrastructure} \approx 8.4 \times 2 = \mathbf{17 \text{ MW}}$$

Running 24/7/365:

$$17 \text{ MW} \times 8,760 \text{ hours} = \mathbf{149 \text{ GWh/year}}$$

That's just *one* AI service from *one* company. Add OpenAI (ChatGPT), Microsoft (Copilot), Anthropic (Claude), Meta (Llama), and dozens of others, each with millions of users. A reasonable estimate for global AI inference might be **500+ GWh/year** and climbing.

For context: Ireland's total electricity consumption is roughly 30,000 GWh/year. AI inference alone could soon rival a small country's entire grid demand.

11.2 The Hardware Reality

A single NVIDIA H200 GPU consumes **700 watts** at full load. To serve a model like Gemini 3 Pro to millions of concurrent users requires thousands of these GPUs running continuously. A typical DGX H200 system (8 GPUs) draws over **10 kilowatts**, enough to power several homes.

System	Power	Capability	Speed
Human brain	~20 W	General intelligence	Real-time
NVIDIA H200 GPU	~700 W	Trillion-param LLMs	ms/token
DGX H200 (8 GPU)	~10 kW	Full LLM inference	ms/token
Arduino Nano	~100 mW	Small SNNs (same as ours)	~100 ms
Our SNN chip	~1 mW	Small SNNs	5 ms

Table 3: Power consumption across computing paradigms. An Arduino Nano *can* run the same 552-synapse network as our chip, but at 100× the power and 20× slower.

Why not just use an Arduino? You could implement our SNN on a microcontroller. The maths is simple enough. But digital computation burns power on every operation: fetching instructions, moving data, clocking flip-flops. Our analog chip computes through physics itself, consuming energy only when spikes occur. For always-on, battery-powered applications, this difference matters.

Looking ahead: This first-generation chip targets a modest 552-synapse network. Future iterations will scale to larger networks, enabling more complex tasks while maintaining the fundamental efficiency advantage of analog neuromorphic computing.

11.3 The Efficiency Gap

The brain achieves general intelligence on 20 watts. Modern AI requires megawatts to approximate narrow aspects of that intelligence. This gap exists because conventional computing architectures waste energy shuffling data between memory and processors, the so-called **von Neumann bottleneck**.

11.4 Compute-in-Memory: The Holy Grail

The holy grail of edge AI is a chip that offers high efficiency, performance, and versatility simultaneously. Compute-in-memory architectures promise exactly this by eliminating data movement entirely.

In conventional processors, data must travel between separate memory and processing units, consuming energy and time. **Compute-in-memory** (also called *in-memory computing* or *processing-in-memory*) performs calculations directly where data is stored.

Our analog SNN chip embodies this principle:

- **Weights are stored as physical properties:** resistances or current source settings
- **Computation happens through physics:** Kirchhoff's laws sum currents automatically
- **No data movement required:** inputs flow through the weight matrix in a single pass
- **Massive parallelism:** all synapses compute simultaneously

This approach has been called the “holy grail” of semiconductor design because it sidesteps the limitations of von Neumann architectures. Major chip manufacturers are pursuing similar ideas, and analog neuromorphic implementations offer strong efficiency gains.

12 Edge AI and Neuromorphic Solutions

Moving AI computation from data centres to edge devices (smartphones, sensors, embedded systems) offers clear benefits, but poses challenges that neuromorphic computing may help solve.

12.1 The Edge Computing Promise

Running AI at the edge means:

- **Lower latency:** No round-trip to the cloud
- **Privacy:** Data stays on-device
- **Reliability:** Works without internet connectivity
- **Reduced bandwidth:** Only results, not raw data, need transmission

12.2 The Power Wall

Edge devices face strict power constraints:

- Smartphones: ~5 W total budget, AI must share with display, radio, etc.
- Wearables: ~100 mW or less
- IoT sensors: Often <10 mW, potentially harvested from environment
- Drones/robots: Battery life directly limits mission duration

Running conventional neural networks under these constraints requires either:

1. Smaller, less capable models
2. Aggressive quantisation (reduced precision)
3. Sparse activation patterns
4. Specialised accelerators

12.3 The Neuromorphic Advantage

Spiking neural networks on analog hardware offer a fundamentally different approach:

Event-driven computation: Unlike conventional chips that compute continuously, neuromorphic hardware only consumes energy when spikes occur. For sparse, real-world signals, this can reduce power by orders of magnitude.

12.3.1 Why Analog?

- **Physics does the maths:** Currents naturally sum (Kirchhoff’s law), capacitors naturally integrate, resistors naturally leak. The differential equations of neural dynamics are solved by physics, not software.

- **No digital overhead:** No clock distribution, no synchronisation, no memory access bottlenecks.
- **Inherent parallelism:** Every neuron computes simultaneously, limited only by physical layout.
- **Low voltage operation:** Analog circuits can operate at lower voltages than digital logic, reducing power quadratically.

12.4 The Tarskii Vision

Our analog SNN chip aims to enable:

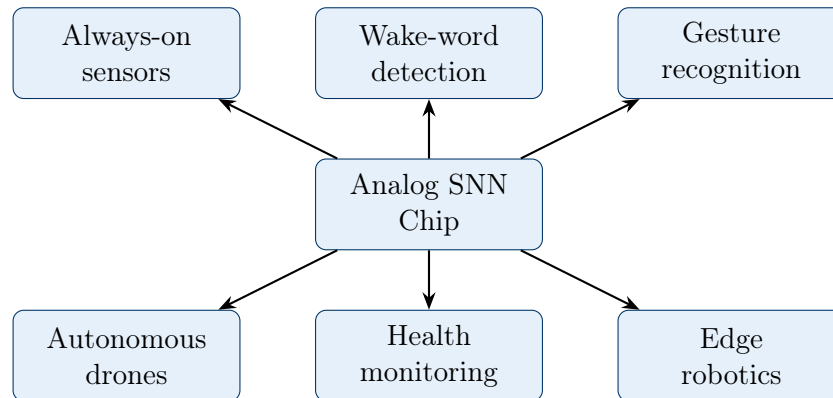


Figure 12: Applications enabled by ultra-low-power neuromorphic computing.

Target specifications:

- Power consumption: ~ 1 mW during active inference
- Latency: **5 ms** for classification (demonstrated in simulation)
- Form factor: Suitable for embedded deployment
- Accuracy: 90.69% on MNIST with only 552 synapses

12.5 From Simulation to Silicon

The path from Gilgamesh simulations to working hardware:

1. **Train** in Gilgamesh (simple mode, fast iteration)
2. **Validate** in Gilgamesh (physics mode, hardware-accurate)
3. **Verify** physics models against ngspice (ensure simulation matches circuit behaviour)
4. **Test discrete components** on stripboard/breadboard (validate individual neuron circuits)
5. **Design PCB** in KiCad (full network layout)
6. **Fabricate PCB** via JLCPCB (prototype board manufacture)
7. **Iterate and refine** based on hardware measurements

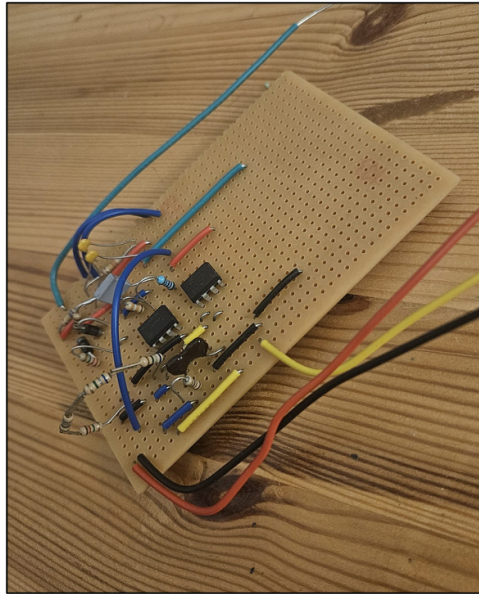


Figure 13: Early prototype: a single LIF neuron built on stripboard for validating our circuit design against SPICE simulations.

This iterative approach, enabled by Gilgamesh’s dual-mode architecture, reduces risk and accelerates development. Each stage provides validation before committing to more expensive fabrication steps.

13 Conclusion

The Tarskii project sits at the intersection of neuroscience, computer science, and electrical engineering. By understanding how biological neurons process information through spikes, we can build artificial systems that capture some of the brain’s efficiency.

Key takeaways:

- **SNNs vs ANNs:** Spiking networks trade continuous values for discrete events, enabling event-driven, low-power computation.
- **LIF neurons:** Simple yet powerful, with a direct mapping to RC circuits.
- **Surrogate gradients:** The key innovation enabling gradient-based training of spiking networks.
- **Gilgamesh:** Our bridge from abstract models to physical hardware.
- **The energy imperative:** Data centres and AI are straining power grids; neuromorphic computing offers a path to sustainable AI.

Looking ahead: As AI spreads into our phones, homes, vehicles, and infrastructure, the need for efficient, low-power neural computation will only grow. Neuromorphic engineering isn’t just an academic curiosity; it may be necessary for a sustainable AI future.